AI ASTROLOGER

Phase 9 — Temporal Dynamics Engine

Revised Roadmap · VedicAstro API Integration

Complete Agent Roadmap · Weeks 31–33

Duration	Depends On	Provides To	API Library
3 Weeks (31–33)	Phase 8 Complete	Phase 10 Divisional Charts	VedicAstro (github.com/diliprk/VedicAstro)

1. What Changed From the Original Roadmap

This document supersedes the original Phase 9 roadmap. The core theoretical model — three-layer temporal synthesis, Vimshottari dasha interpretation logic, transit analysis, and the TemporalPrediction output structure — is unchanged. What changes is the implementation strategy for the two most calculation-intensive deliverables: dasha timeline computation and planetary transit positions.

The VedicAstro open-source library provides a tested, production-ready implementation of the Vimshottari dasha calculator (compute_vimshottari_dasa) and chart generation (generate_chart, get_planets_data_from_chart, get_houses_data_from_chart). Adopting these functions eliminates approximately four days of low-level calculation work in Week 31 and replaces it with integration, validation, and adapter work. The time saved is reinvested into deeper calibration, historical validation, and the transit synthesis layer — the parts of Phase 9 that produce the most interpretive value.

Scope of Change — What Stays vs What Changes

UNCHANGED: Three-layer temporal model, dasha interpretation logic, yoga activation detection, transit interaction types, temporal synthesiser architecture, TemporalPrediction object schema, Neo4j schema, success criteria, and all of Weeks 32–33. CHANGED: Week 31 implementation approach — dasha calculation is delegated to VedicAstro's compute_vimshottari_dasa instead of built from scratch. An adapter layer is introduced to normalise VedicAstro output into the project's canonical schema.

2. VedicAstro API — Functions Used, Functions Rejected

The VedicAstro library provides seven functions. Not all of them are appropriate for this project. The decision on each function is based on compatibility with the Parashari Jyotish tradition that the entire reasoning engine (Phases 5–8) is built on.

Function	Use?	Reason	Replaces
generate_chart	YES	Wraps flatlib correctly with Lahiri ayanamsa for sidereal positions. Provides the flatlib.Chart object used by all other functions.	Phase 4 Swiss Ephemeris chart generation — but must be validated for ayanamsa correctness first
get_planets_data_from_chart	YES	Returns planet signs, degrees, houses, dignity states, retrograde status in structured format. Parashari-compatible output.	Phase 4 planet position calculations and dignity assignments
get_houses_data_from_chart	YES	Returns Whole Sign house assignments — the correct house system for Jyotish. Compatible with Phase 5–8 reasoning.	Phase 4 house cusp calculations

Function	Use?	Reason	Replaces
compute_vimshottari_dasa	YES	Computes mahadasha, antardasha, and pratyantardasha timelines with dates. Directly replaces Phase 9 Week 31 Days 1–4.	dasha_calculator.py Days 1–4 of original Week 31
get_planet_wise_significators	NO	KP (Krishnamurti Paddhati) sub-lord significators. Different tradition from Parashari. Would conflict with Phase 5–8 house lord assignments.	Not replaced — not needed
get_house_wise_significators	NO	KP significators for houses. Same reason as above — incompatible with Parashari reasoning engine.	Not replaced — not needed
get_planetary_aspects	NO	Western astrology aspects (trine, sextile, square, opposition). Phase 7 uses Jyotish drishti (house-based). These are different systems and cannot be substituted.	Not replaced — Phase 7 drishti stays as built

Critical: Ayanamsa Verification Before Any Use

The VedicAstro library wraps flatlib, which defaults to the tropical zodiac. Jyotish requires the sidereal zodiac using Lahiri ayanamsa. Before using any VedicAstro output in production, verify on Day 1 of Week 31 that generate_chart is correctly applying the Lahiri ayanamsa. Test by computing the chart for a known birth date, comparing the Moon's sidereal position against a trusted Jyotish software output. If the positions differ by approximately 23–24 degrees, the ayanamsa is not being applied and VedicAstro cannot be used for chart generation. In that case, revert to direct Swiss Ephemeris for generate_chart, get_planets_data_from_chart, and get_houses_data_from_chart — but compute_vimshottari_dasa can still be used if given a correctly computed Moon nakshatra position as input.

3. The Adapter Layer — Bridging VedicAstro to the Project Schema

VedicAstro returns data in its own format — pandas DataFrames, flatlib objects, and string-based field names that do not match the project's canonical entity names established in Phase 1. The adapter layer is the single most important new piece of work introduced by this revised roadmap. It sits between VedicAstro's output and the project's Neo4j schema, and it performs three transformations.

3.1 Canonical Name Normalisation

VedicAstro uses its own planet and sign names which may differ from the canonical forms established in the Phase 1 ontology. Every entity name coming out of VedicAstro must pass through the Phase 1 normalise() function before being stored in Neo4j or consumed by the

reasoning engine. This is non-negotiable — the entire Phase 5–8 system is built on canonical names, and any deviation creates silent data corruption.

Expected normalisation mappings to verify and handle:

- Planet names: verify that VedicAstro's planet labels ('Sun', 'Moon', 'Mars' etc.) map correctly to the project canonicals (SUN, MOON, MARS). Pay special attention to Rahu and Ketu — some libraries call them 'Dragon Head' / 'Dragon Tail' or 'North Node' / 'South Node'.
- Sign names: verify 'Aries', 'Taurus' etc. map to ARIES, TAURUS etc. Check that the library uses the same 12-sign list — some sidereal systems use slightly different boundary degrees.
- House numbers: VedicAstro returns house numbers as integers. Verify the numbering convention (1-indexed from Lagna) matches the project's HOUSE_1 through HOUSE_12 schema.
- Dignity labels: VedicAstro may return dignity as 'Exalted', 'Debilitated', 'Own', 'Friendly', 'Enemy', 'Moolatrikona'. Map these to the exact string values expected by Phase 6's dignity scoring lookup tables.

3.2 Dasha Output Schema Mapping

The compute_vimshottari_dasa function returns dasha data in a tabular format. The adapter must transform this into the DashaperiodNode schema used by the project's Neo4j graph. The key fields to map are:

VedicAstro Output Field	Project Schema Field	Notes
Mahadasha planet name	DashaperiodNode.planet	Must be normalised via normalise()
Mahadasha start date	DashaperiodNode.start_date	Verify date format — convert to ISO 8601
Mahadasha end date	DashaperiodNode.end_date	Verify date format — convert to ISO 8601
Antardasha planet name	DashaperiodNode.planet	Child node, must be normalised
Antardasha start/end dates	DashaperiodNode.start/end_date	Same format conversion
Pratyantardasha planet name	DashaperiodNode.planet	Grandchild node, normalised
Dasa type level	DashaperiodNode.dasha_type	'mahadasha', 'antardasha', 'pratyantardasha'

3.3 Missing Fields That Must Be Added After Parsing

VedicAstro's dasha output contains dates and planet names. It does not contain the interpretation fields that the temporal synthesiser needs. The adapter must compute and attach the following fields after parsing the VedicAstro output, using data already stored in Neo4j from Phases 6–8:

- activation_weight: retrieved by looking up the dasha planet's strength_total from Phase 6 ChartPlanet nodes, normalised to 0–1.
- yogas_activated: retrieved by querying Phase 7 DetectedYoga nodes for yogas whose participant planets include the dasha planet.
- parent_period_id: the Neo4j node ID of the parent mahadasha or antardasha node, linking the hierarchy together.
- house_rulership: the houses this planet rules for the chart's lagna, retrieved from functional_nature.json via Phase 6 output.

4. The Three-Layer Temporal Model

This section is unchanged from the original roadmap. It is reproduced here for completeness so this document is self-contained.

Layer	What It Represents	Changes How Often
Layer 1 — Natal	The permanent structure of the chart: what themes are possible, which planets are strong, which houses are dominant (Phase 8 output). This never changes.	Never — fixed at birth
Layer 2 — Dasha	The planetary periods that determine which natal planet is running the show. A strong Mars in the 10th house only produces career results when Mars is active in the dasha cycle. NOW PROVIDED BY VedicAstro compute_vimshottari_dasa.	Every few months to years
Layer 3 — Transit	The current positions of planets in the sky, stimulating natal houses and planets from outside. Transit positions use Swiss Ephemeris directly — NOT the VedicAstro get_planetary_aspects function, which is Western aspects only.	Daily to yearly

Transit Layer Uses Swiss Ephemeris Directly — Not VedicAstro Aspects

This is the most important boundary to keep clear. VedicAstro's get_planetary_aspects returns Western aspects (trine, sextile, square, conjunction). Phase 9's transit analysis requires Jyotish drishti — house-based aspects where every planet aspects the 7th house from its position, with Mars, Jupiter, and Saturn having special additional aspects. These are fundamentally different systems. Transit position data (where planets are on a given date) comes from Swiss Ephemeris via pyswisseph. The Jyotish aspect evaluation logic is implemented in the transit_calculator.py module using the same drishti rules from Phase 7.

5. Revised Deliverables

5.1 vedic_astro_adapter.py — New Deliverable

A new module that sits between the VedicAstro library and the rest of the project. This module owns all interaction with VedicAstro — no other module imports VedicAstro directly. It exposes three clean functions to the rest of the system:

- get_chart_data(birth_date, birth_time, latitude, longitude, timezone): calls generate_chart, get_planets_data_from_chart, and get_houses_data_from_chart, normalises all entity names through normalise(), and returns a structured ChartData object in the project's canonical schema.
- get_dasha_timeline(chart_data): calls compute_vimshottari_dasa with the chart's Moon nakshatra data, parses the output, normalises planet names, and returns a list of DashaperiodNode objects ready for Neo4j ingestion.
- verify_ayanamsa(chart_data, reference_positions): a validation function used during testing only. Compares VedicAstro's planet positions against a reference set and raises an error if the Moon's position differs by more than 0.5 degrees. Must pass before any chart data is used in production.

5.2 dasha_calculator.py — Simplified Deliverable

In the original roadmap, this module built the Vimshottari dasha calculation engine from scratch. With VedicAstro, this module becomes an orchestration and enrichment layer rather than a calculation engine. Its responsibilities are: calling vedic_astro_adapter.get_dasha_timeline(), attaching the interpretation fields (activation_weight, yogas_activated, house_rulership) to each DashaperiodNode, storing the complete dasha chain in Neo4j with correct parent-child relationships, and exposing query functions for the temporal synthesiser to consume.

5.3 transit_calculator.py — Unchanged Deliverable

This module is built from scratch using Swiss Ephemeris (pyswisseph) exactly as specified in the original roadmap. It computes current planetary positions for any date, evaluates all four Jyotish transit interaction types (transit over natal planet, transit over natal house, transit aspect using drishti rules, and double transit detection), computes transit strength scores, and stores TransitEvent nodes in Neo4j. VedicAstro is not used here.

5.4 temporal_synthesiser.py — Unchanged Deliverable

The core synthesis module is architecturally unchanged. It consumes the dasha timeline (now populated via VedicAstro adapter rather than custom calculation) and the transit data (from Swiss Ephemeris) and produces TemporalPrediction objects. The synthesis formula, strength band classifications, yoga activation detection, and window scanner are all unchanged from the original roadmap.

6. Revised Week-by-Week Task Plan

Week 31 is significantly restructured compared to the original roadmap. Weeks 32 and 33 are unchanged except for one addition to Week 33 Day 3 where the adapter's normalisation accuracy is re-verified on the full test set.

Week 31 — VedicAstro Integration and Dasha Pipeline

Day 1 — Library Installation and Ayanamsa Verification

Install VedicAstro and its dependencies (flatlib, pandas). Run the ayanamsa verification test: generate a chart for at least three known birth dates and time-location combinations where the correct sidereal planet positions are known from a trusted Jyotish software (Jagannatha Hora, Kala, or similar). Compare the Moon's sidereal sign and degree from VedicAstro against the reference. The Moon is the most critical planet to verify because the entire dasha entry point depends on the Moon's nakshatra position. If positions match within 0.5 degrees: proceed with VedicAstro for chart generation. If they do not: document the discrepancy, check whether the ayanamsa parameter is being passed correctly to generate_chart, and escalate to Orchestrator if the issue cannot be resolved. Do not proceed to Day 2 without passing this verification.

Day 2 — Adapter Module: Chart Data Functions

Build vedic_astro_adapter.py. Implement get_chart_data() wrapping generate_chart, get_planets_data_from_chart, and get_houses_data_from_chart. Implement the full normalisation layer for all entity names. Test with 10 birth charts: verify that all planet signs, houses, dignity states, and retrograde flags match the Phase 4 ChartPlanet nodes already in Neo4j for those charts. If any discrepancy exists, it indicates a normalisation gap or ayanamsa issue — resolve before proceeding. The adapter's output must be byte-for-byte identical in schema to what Phase 4 produces so that Phase 5–8 are not affected.

Day 3 — Adapter Module: Dasha Timeline Function

Implement get_dasha_timeline() in the adapter. Call compute_vimshottari_dasa with the chart's Moon nakshatra and birth date. Parse the returned DataFrame or table structure into a list of DashaperiodNode objects. Normalise all planet names. Apply ISO 8601 date formatting. Verify the complete mahadasha timeline for 5 test charts against a trusted Jyotish software: all nine mahadasha start and end dates must match exactly. Verify antardasha dates for the current running mahadasha. Verify at least one pratyantardasha chain. Zero tolerance for date discrepancies — if compute_vimshottari_dasa produces wrong dates, identify whether it is a parsing error in the adapter or a bug in the library itself.

Day 4 — Dasha Enrichment: Attaching Interpretation Fields

Implement dasha_calculator.py's enrichment layer. For each DashaperiodNode returned by the adapter, query Neo4j to retrieve: the planet's strength_total from Phase 6 and compute activation_weight, the planet's functional nature and house rulership for this lagna from functional_nature.json, and all Phase 7 DetectedYoga nodes where this planet is a participant — these become the yogas_activated list. Store the fully enriched dasha chain in Neo4j with correct parent-child CONTAINS relationships. Test with 10 charts: verify that activation_weight values are higher for planets with strong Phase 6 scores and lower for weak planets.

Day 5 — Yoga Activation Window Detection and Week 31 Integration

Implement the yoga activation detection logic from Section 5.3 of the original theoretical roadmap: cross-reference the dasha timeline against DetectedYoga nodes to identify all windows where the mahadasha and antardasha planets match yoga participant planets. Flag these periods as high-significance yoga activation windows on the DashaperiodNode objects. Run end-to-end integration for 20 charts: from birth data through VedicAstro adapter, through enrichment, to fully populated Neo4j dasha chain with yoga activation flags. Verify Neo4j query performance is under 100ms for retrieving a chart's full dasha chain.

Week 32 — Transit Calculator and Synthesis

Day 1 — Swiss Ephemeris Transit Position Calculator

Build transit_calculator.py using pyswisseph directly — no VedicAstro involvement. For any given date, compute the sidereal positions of all nine planets using the Lahiri ayanamsa. The ayanamsa value must be the same constant used in Phase 4 chart calculations — enforce this through a shared configuration constant, not independent values. Verify transit positions for 5 reference dates against known ephemeris data within 0.1 degree precision.

Day 2 — Transit Interaction Detection

Implement the four Jyotish transit interaction types using the drishti rules from Phase 7 — not VedicAstro's Western aspects. Transit over natal planet: transiting planet in same sign as natal planet. Transit over natal house: transiting planet in a house that received a high Phase 8 importance score. Transit aspect: using Jyotish drishti (7th aspect for all planets, special aspects for Mars, Jupiter, Saturn). Double transit: Jupiter and Saturn simultaneously in a 1-7 axis. Compute transit strength scores and store TransitEvent nodes in Neo4j.

Day 3 — Temporal Synthesiser Core

Implement temporal_synthesiser.py. Connect the three layers: retrieve active dasha stack from Neo4j dasha chain (originally computed via VedicAstro, now enriched with interpretation fields), retrieve active transit events above the 0.3 threshold, retrieve yoga activation flags from dasha nodes. Apply the four-step synthesis process and produce TemporalPrediction objects. Synthesis weights: dasha layer 50%, transit layer 30%, yoga activation layer 20% — stored as named constants.

Day 4 — Window Scanner and Event Templates

Implement the forward-looking window scanner that iterates upcoming antardashas and scores them by theme relevance. Implement the six event-type templates (marriage, career change, major wealth gain, foreign travel, spiritual turning, health challenge). Test with five different query types across 5 charts.

Day 5 — Neo4j Storage and Integration Test

Store all transit events and TemporalPrediction objects for the 100 test chart set. Run cross-layer integration queries combining Phase 8 house importance scores with Phase 9 dasha activation weights. Verify end-to-end pipeline for 20 charts and review output quality.

Week 33 — Validation, Calibration, and Handoff

Day 1 — Historical Validation

For 5 public figures with documented life events and known birth data, run the temporal engine over historical periods and verify that high-period-strength windows correspond to documented major life events. Measure accuracy rate.

Day 2 — Synthesis Weight Calibration

Based on historical validation, adjust the three synthesis layer weights if needed. Re-run the historical validation set. Document the final weights with reasoning. All weights remain named constants.

Day 3 — Adapter Normalisation Full-Set Verification

This is a new day added in this revised roadmap. Run the VedicAstro adapter against all 100 test charts and verify that every entity name in every output has been successfully normalised to its canonical form. Check that no unknown entity names are slipping through the normalise() function. Any unmapped entity names must be added to the Phase 1 synonym map before handoff. This is the quality gate that ensures the adapter layer has not introduced any data inconsistencies into the 100-chart test set.

Day 4 — Phase 10 and Phase 11 Interface Documentation

Define query interface contracts for Phase 10 (Divisional Charts) and Phase 11 (NLG). Confirm that TemporalPrediction reasoning_steps field contains sufficient information for Phase 11 to generate natural language without additional Neo4j queries. Write the interface specification documents.

Day 5 — Full Validation and Handoff Preparation

Run the complete success criteria checklist against the 100-chart test set. Prepare the handoff package including all four module public interfaces, the adapter normalisation verification report, Neo4j schema documentation, calibration decisions log, and performance benchmarks. Notify Orchestrator that Phase 9 is ready for review.

7. Neo4j Schema Updates

The Neo4j schema introduced by Phase 9 is unchanged from the original roadmap. The VedicAstro adapter produces data that conforms to the same node and relationship definitions. The three node types remain:

7.1 DashaperiodNode

Property	Type & Description
dasha_type	String — 'mahadasha', 'antardasha', or 'pratyantardasha'
planet	String — canonical planet name, normalised via normalise()
start_date	Date — ISO 8601 period start date
end_date	Date — ISO 8601 period end date
duration_days	Integer — period length in days
activation_weight	Float 0–1 — dasha planet's strength from Phase 6, enriched by adapter
yogas_activated	String list — yoga names activated during this period, enriched by adapter
parent_period_id	String — Neo4j ID of parent mahadasha or antardasha node
house_rulership	Integer list — houses this planet rules for the chart's lagna
source	String — 'vedic_astro_api' to track data provenance

7.2 TransitEvent Node

Property	Type & Description
transiting_planet	String — canonical planet name
transit_type	String — 'conjunction', 'aspect', 'double_transit'
natal_target	String — natal house number or planet name

Property	Type & Description
start_date	Date — transit begins
end_date	Date — transit ends
transit_strength	Float 0–1 — computed transit strength score
significance	String — 'low', 'moderate', 'high', 'very_high'
aspect_type	String — Jyotish drishti type: '7th', '4th', '8th', '5th', '9th', '3rd', '10th'

7.3 Relationships

Relationship	Direction & Meaning
(Chart)-[:HAS_DASHA]->(DashaperiodNode)	Links chart to its mahadasha nodes
(DashaperiodNode)-[:CONTAINS]->(DashaperiodNode)	Links mahadasha to its antardasha children
(DashaperiodNode)-[:GOVERNED_BY]->(Planet)	Links period to the governing Planet node
(Chart)-[:HAS_TRANSIT_EVENT]->(TransitEvent)	Links chart to significant transit events
(TransitEvent)-[:ACTIVATES]->(HouseInfluence)	Links transit to the house influence node it stimulates

8. Success Criteria

Phase 9 is complete when all of the following criteria are met. The ayanamsa verification criterion is new in this revised roadmap:

Criterion	Measurement	Pass Threshold
Ayanamsa verification	VedicAstro planet positions vs trusted Jyotish software	Moon position within 0.5 degrees on all 3 reference charts — must pass before any other work begins
Adapter normalisation completeness	Zero unknown entity names across 100-chart test set	100% — every entity name normalised via Phase 1 normalise()
Dasha calculation accuracy	Mahadasha and antardasha dates vs professional software	100% match on all 10 reference charts
Transit position accuracy	Transit positions from pyswisseph vs known ephemeris	Within 0.1 degrees on all 5 reference dates

Criterion	Measurement	Pass Threshold
Historical validation accuracy	Correct period identification for known life events	80%+ agreement across 5 historical charts
Yoga activation detection	All relevant yoga windows identified in dasha chain	Zero missed activations for known yoga-dasha overlaps
Window scanner plausibility	Top returned windows reviewed by expert	90%+ rated plausible for 5 query types
Neo4j storage completeness	All dasha and transit nodes stored with correct provenance	Zero missing records; all source fields set to 'vedic_astro_api' for dasha nodes
TemporalPrediction completeness	All fields populated including reasoning_steps	Zero null fields in any TemporalPrediction object
Performance — dasha pipeline	VedicAstro call + adapter enrichment + Neo4j write	Under 500ms per chart
Performance — transit computation	Single-date transit analysis via pyswisseph	Under 300ms per chart
Performance — full synthesis	End-to-end TemporalPrediction generation	Under 500ms per chart
Orchestrator review	Architecture compliance and interface approval	Approved before Phase 10 begins

9. Integration Interfaces

9.1 Inputs Consumed by Phase 9

Input	Source & Description
Birth date, time, latitude, longitude, timezone	User input — passed to VedicAstro generate_chart via the adapter
Moon nakshatra and degree at birth	Extracted from VedicAstro get_planets_data_from_chart output by adapter
ChartPlanet nodes with strength scores	Phase 6 — used by dasha enrichment to compute activation_weight
functional_nature.json per lagna	Phase 6 — house rulership and functional benefic/malefic assignments for dasha enrichment
DetectedYoga nodes	Phase 7 — yoga participant planets for yoga_activated field enrichment
HouseInfluence nodes	Phase 8 — house importance scores for transit strength weighting
DominantTheme nodes	Phase 8 — theme rankings for prediction prioritisation

Input	Source & Description
pyswisseph (Swiss Ephemeris)	External — real-time transit position data; used independently from VedicAstro

9.2 Outputs Provided by Phase 9

Output	Consumed By
DashaperiodNode chain in Neo4j	Phase 10 Divisional Charts — dasha timing for D9/D10 cross-reference; Phase 11 NLG — temporal framing in interpretation
TransitEvent nodes in Neo4j	Phase 11 NLG — transit descriptions in prediction paragraphs
TemporalPrediction objects	Phase 11 NLG — complete structured input for temporal language generation
Window scanner query function	Phase 11 NLG — answers 'when will X happen?' queries at runtime
Yoga activation windows	Phase 11 NLG — highest-value temporal prediction outputs
vedic_astro_adapter.py (public API)	Phase 10 — may reuse chart generation functions for divisional chart base data

10. Risk Register

Risk	Likelihood	Impact	Mitigation
Ayanamsa not correctly applied in VedicAstro — all planet positions wrong	Medium	High	Day 1 ayanamsa verification is a hard gate. If it fails and cannot be resolved, use pyswisseph for chart generation and only use compute_vimshottari_dasa from VedicAstro, passing Moon nakshatra data computed independently.
compute_vimshottari_dasa returns wrong date formats or unexpected structure	Medium	Medium	On Day 3, print and inspect the raw VedicAstro output before writing any parsing code. The adapter should fail loudly on unexpected structure — use assertions, not silent defaults.
VedicAstro planet name conventions differ from Phase 1 canonical names	High	Medium	The normalise() function already handles synonym mapping. Add a dedicated test on Day 2 that passes every VedicAstro entity name through normalise() and verifies zero unknowns. Any unmapped names go to the Phase 1 ontology team immediately.

Risk	Likelihood	Impact	Mitigation
Library dependency conflicts with existing project dependencies	Low	Medium	Install in a clean virtual environment on Day 1 and verify no conflicts before any integration work begins.
VedicAstro compute_vimshottari_dasa antardasha dates slightly different from professional software	Medium	Medium	If dates are off by 1–2 days, investigate whether the discrepancy is in the Julian day conversion. If off by more, check whether the library uses a different nakshatra-to-dasha mapping convention. Document and escalate to Orchestrator if unresolvable.
KP significator functions accidentally used by future agents	Low	High	vedic_astro_adapter.py deliberately does not expose get_planet_wise_significators or get_house_wise_significators. They are not imported. Add a comment at the top of the adapter module explicitly documenting which VedicAstro functions are excluded and why.

11. Orchestrator Agent Notes

Architecture Compliance Requirements

All VedicAstro interactions must go through vedic_astro_adapter.py only — no other module may import VedicAstro directly. All entity names from VedicAstro must pass through normalise() before any downstream use. The ayanamsa constant must be shared between Phase 4, Phase 9's VedicAstro adapter, and Phase 9's Swiss Ephemeris transit calculator — never set independently in three places. The adapter's source field on all DashaperiodNode objects must be set to 'vedic_astro_api' to maintain data provenance. Phase 9 must not use VedicAstro's get_planetary_aspects under any circumstances — Jyotish drishti rules from Phase 7 are the only aspect system used in this project.

Weekly reporting to Orchestrator should include: ayanamsa verification result from Day 1 with evidence (planet position comparison table), adapter normalisation accuracy on test charts, dasha date accuracy versus professional software, historical validation accuracy, any VedicAstro library issues encountered and their resolutions, and synthesis weight calibration decisions.

Handoff to Phase 10 is blocked until: ayanamsa verification has passed and is documented, adapter normalisation shows zero unknown entity names across the 100-chart test set, dasha accuracy is 100% verified, historical validation meets the 80% threshold, and the Phase 10 and Phase 11 interface contracts are signed off by the Orchestrator.